Client–Server Chat Application

C# / .NET 9 — TCP sockets, threads, delegates and events

LAUNAY-SCHAPMAN Alexis & AUGAY Eliott — EFREI I2, Interface Development and Design

Table of contents

a. Functions implemented
b. Design of each feature
    1. Architecture overview
    2. Start and stop the server
    3. Connection and username login
    4. Public messages and broadcast with delegates
    5. Private messages
    6. Commands /users, /name, /help, /quit
    7. Logging server activity
    8. Disconnections and client robustness
    9. Extension: /joke
    10. Known limitations
c. Main parts of the C# implementation

a. Functions implemented

Numbering follows the project statement (course handout, not included in this repository). All listed functions are implemented.

Server-side functions

#	Function	Implementation
1	Start and stop the server	Accept loop in its own thread; typing `stop` in the server console warns every client, stops the listener and closes all sockets.
2	Accept multiple client connections via TCP sockets	`TcpListener` on 127.0.0.1:5050, one background thread per client (`ClientHandler`).
3	Authenticate users (username-based login)	First line sent by the client is the username; it must be unique (case-insensitive) and valid, otherwise the server asks again.
4	Maintain a list of connected clients	`Dictionary<string, ClientHandler>` in `Server`, protected by a lock on a private object.
5	Receive messages from clients	Text protocol, one line = one message, read by the client's thread; commands are parsed on the server.
6	Broadcast messages using delegates	`MessageEventHandler` delegate + `MessageBroadcast` event; every `ClientHandler` subscribes. Messages are time-stamped by the server: `[HH:mm] user : text`.
7	Send private messages to specific users	`/msg user text`: written directly to the recipient's writer, echoed to the sender; clear error if the user is unknown.
8	Log server activity	`Logger` subscribed to the server events `ClientConnected`, `ClientDisconnected`, `MessageReceived`, `ErrorOccurred`; writes to the console and `server.log`.

Client-side functions

#	Function	Implementation
1	Connect to the server	`TcpClient` to 127.0.0.1:5050; clear message if the server is not running.
2	Send username	The first line typed is sent as the username; the server's answer is displayed.
3	Send public messages	Keyboard thread (main thread) writes each line to the socket.
4	Receive and display broadcast messages	Separate network-reading thread prints every line received.
5	Send and receive private messages	`/msg user text`; private messages are shown as `(privé)`.

Commands

Command	Effect
`/name new`	Change username (uniqueness checked under the lock), announced to everyone.
`/quit`	Clean disconnection; departure announced to everyone.
`/users`	List connected users.
`/msg user text`	Private message.
`/help`	List of commands (extension).
`/joke`	Fetches a programming joke from api.chucknorris.io (category `dev`) and broadcasts it (extension).

Extensions beyond the statement

- Server-side timestamps, arrival/departure announcements, `/help`, `/joke`.
- Robust cleanup: a client that disconnects abruptly is always unsubscribed and removed from the list.
- Persistent log file `server.log`.

Note: user-facing messages of the application are in French (e.g. « a rejoint le chat » = “joined the chat”, « privé » = “private”).

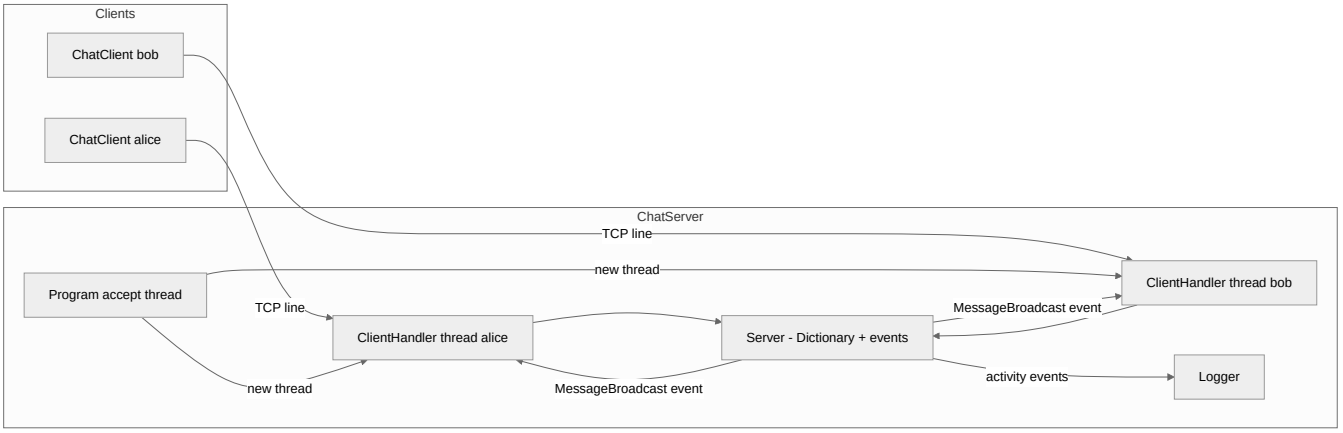
b. Design of each feature

b.1 Architecture overview

Two console projects in one solution (`ChatApp.sln`):

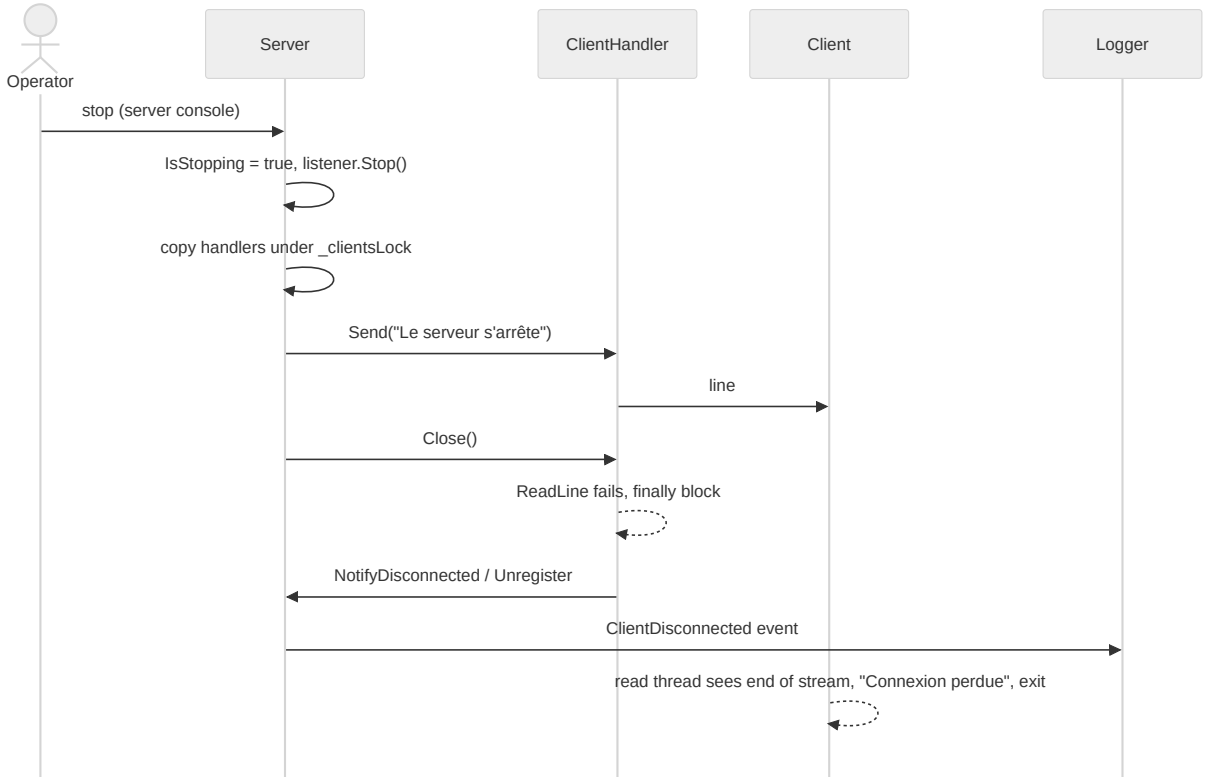
- **ChatServer**: `Program.cs` (listener, accept thread, console `stop`), `Server` (shared object: client dictionary, broadcast event, activity events), `ClientHandler` (one per client, runs in its own thread, parses commands), `Logger` (subscriber to activity events), `JokeService` (HTTP call).
- **ChatClient**: `Program.cs` with two threads — main thread reads the keyboard and writes to the socket, a background thread reads the socket and prints.

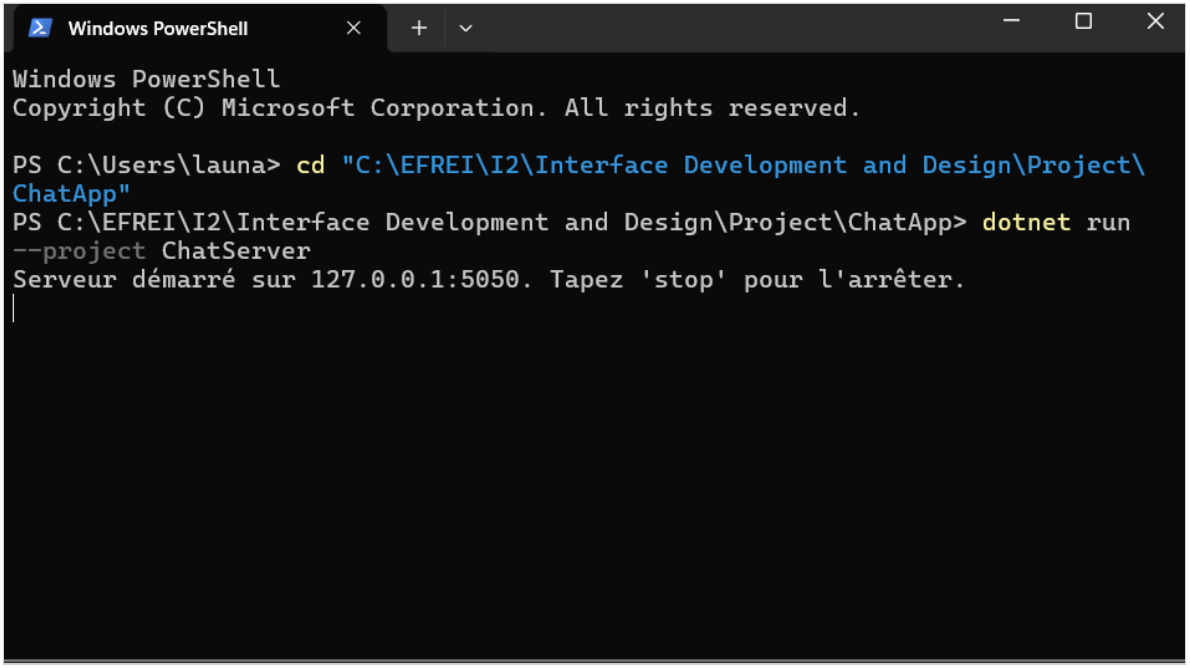
The protocol is plain text over TCP, one line per message (UTF-8). The client is "dumb": it only sends what is typed and displays what is received; all parsing happens on the server. A single `Server` instance is shared by all `ClientHandler` threads; shared state (the dictionary, each writer, the log file) is protected by locks.



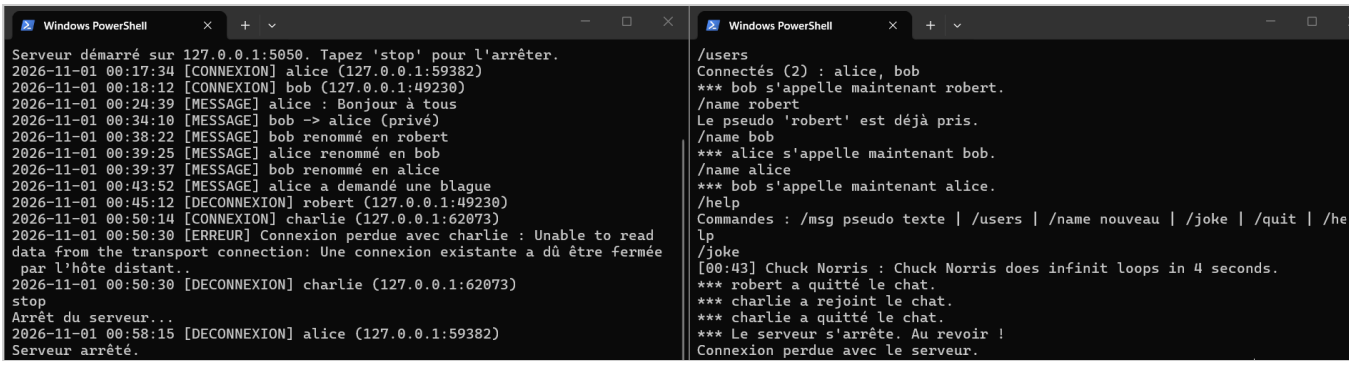
b.2 Start and stop the server (server function 1)

The accept loop runs in a dedicated background thread so that the main thread stays free to read the server console. When the operator types `stop`, the server sets `IsStopping`, stops the listener (which unblocks `AcceptTcpClient` with a `SocketException`), copies the list of handlers under the lock, then warns and closes each client *outside* the lock. Each handler's `finally` then unregisters itself; the main thread waits until the list is empty before exiting.





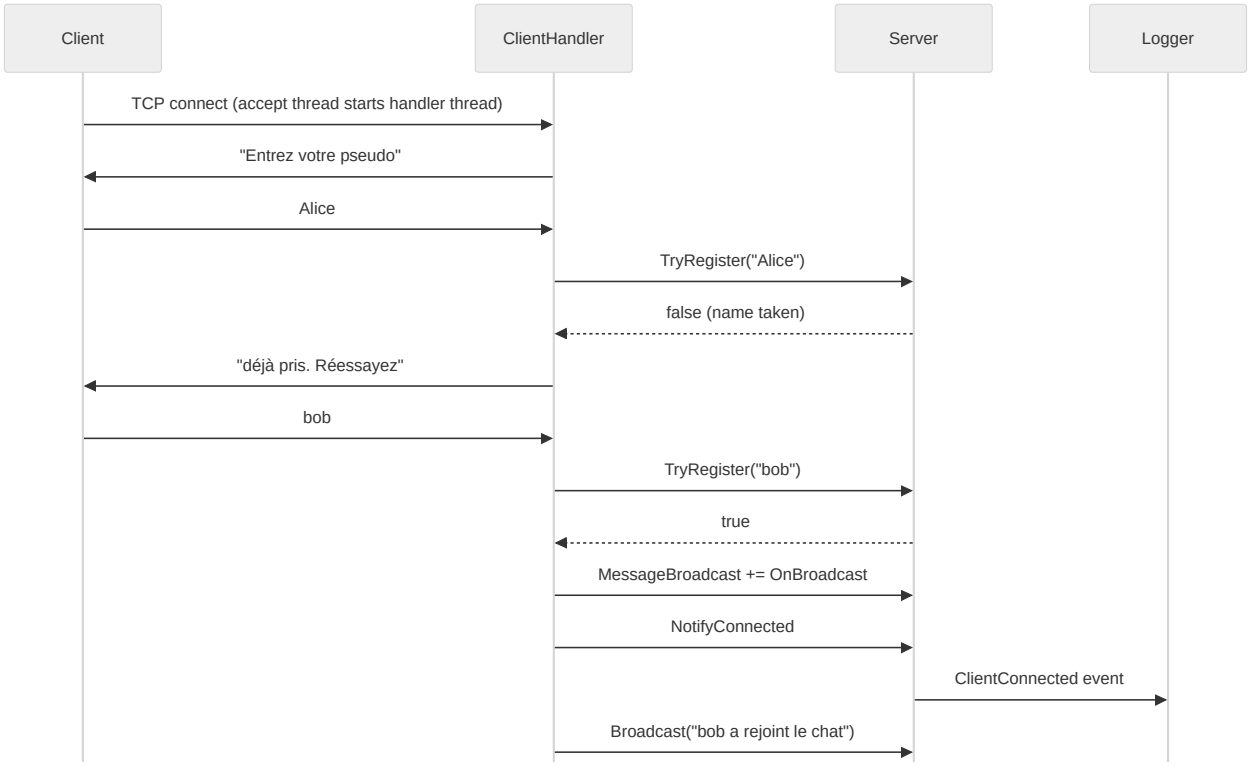
Server started

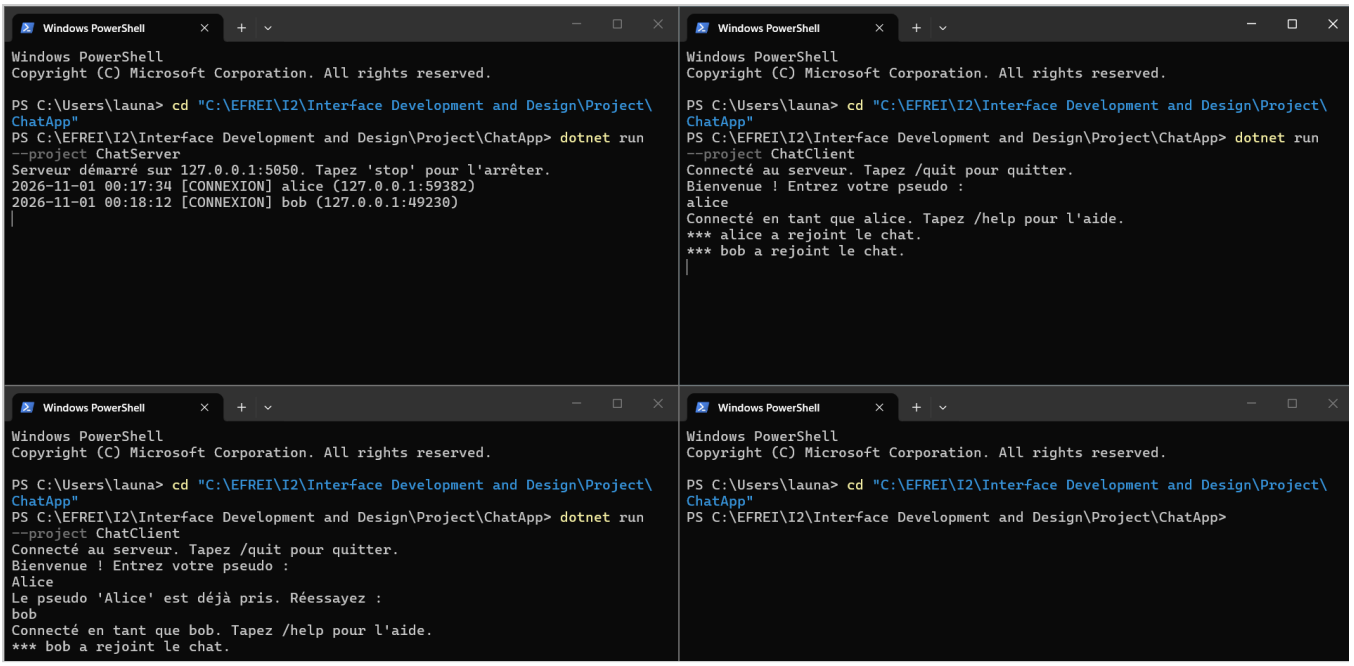


Typing `stop`: clients are warned and disconnected

b.3 Connection and username login (server 2, 3, 4 — client 1, 2)

For each accepted `TcpClient` the accept thread creates a `ClientHandler` and starts it in a new background thread. The handler asks for a username; `Server.TryRegister` checks uniqueness and inserts into the dictionary in a single locked operation, so two clients cannot obtain the same name. If the name is taken or invalid, the server asks again. Only after a successful login does the handler subscribe to the broadcast event and announce the arrival.

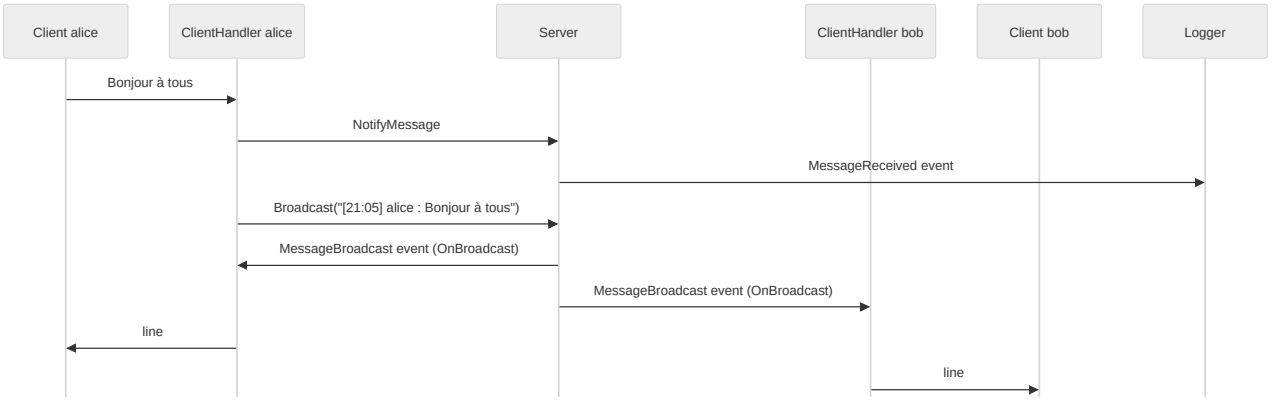


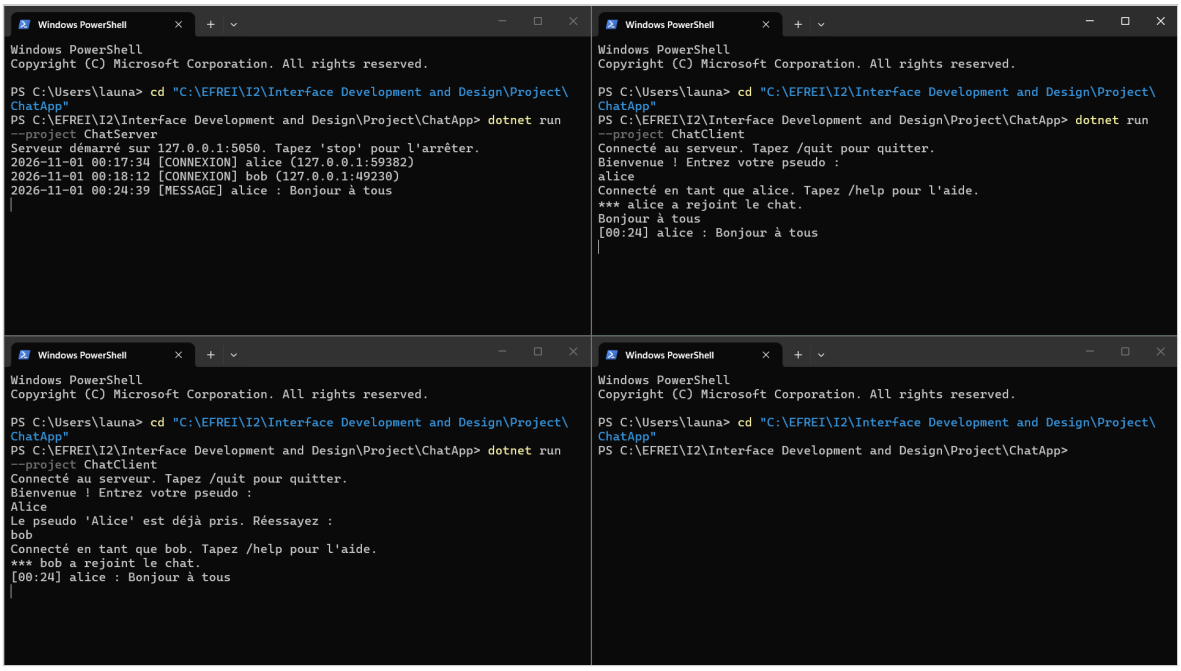


Duplicate username refused, then successful login

b.4 Public messages and broadcast with delegates (server 5, 6 — client 3, 4)

A line that does not start with `/` is a public message. The handler formats it with a server-side timestamp and calls `Server.Broadcast`, which raises the `MessageBroadcast` event. The event invokes the `OnBroadcast` method of every subscribed handler; each one writes the line to its own client under its own writer lock. The server never iterates over clients to send a message: the delegate invocation list does it, which keeps `Server` decoupled from `ClientHandler`.

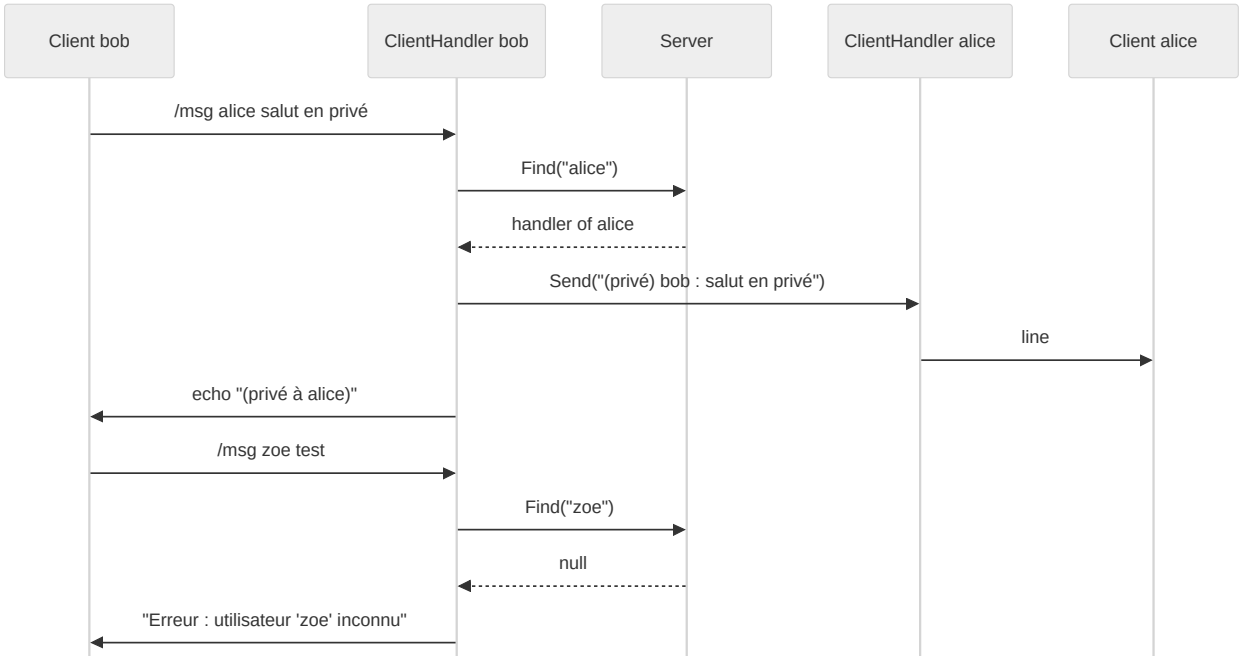


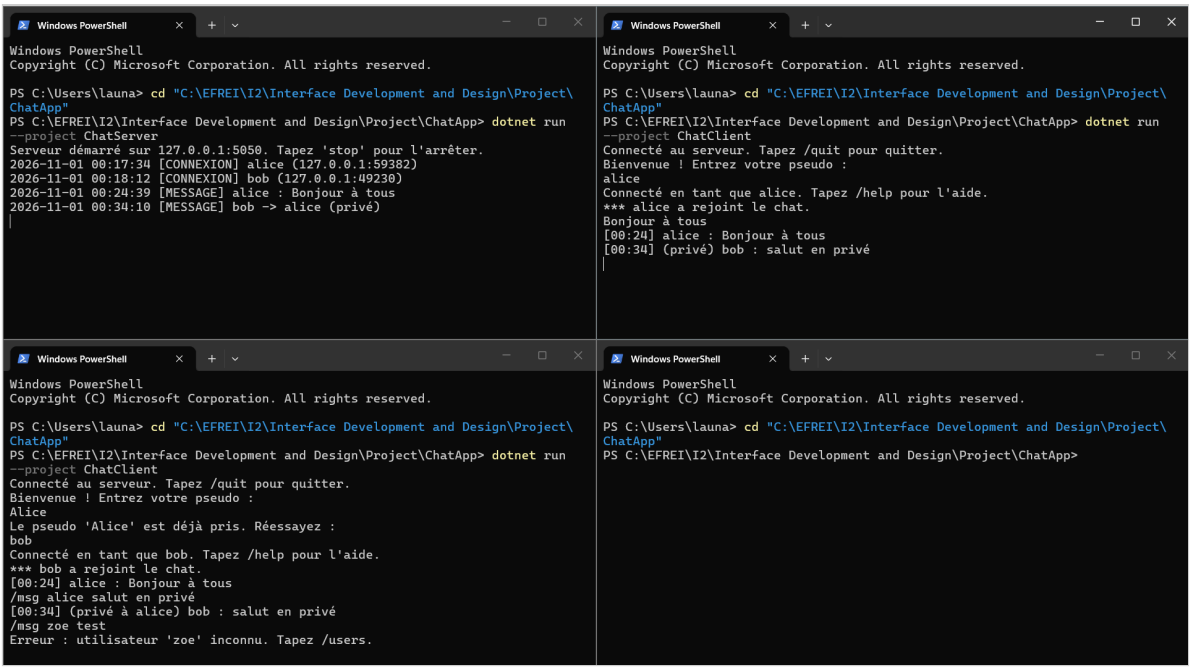


Message broadcast to every client with a server timestamp

b.5 Private messages (server 7 — client 5)

`/msg user text` is parsed by the sender's handler. `Server.Find` looks the recipient up in the dictionary under the lock; the handler then writes directly into the recipient's writer (`target.Send`, protected by the recipient's writer lock) and echoes the message to the sender. The broadcast event is not used, so nobody else receives it. An unknown user produces a clear error.

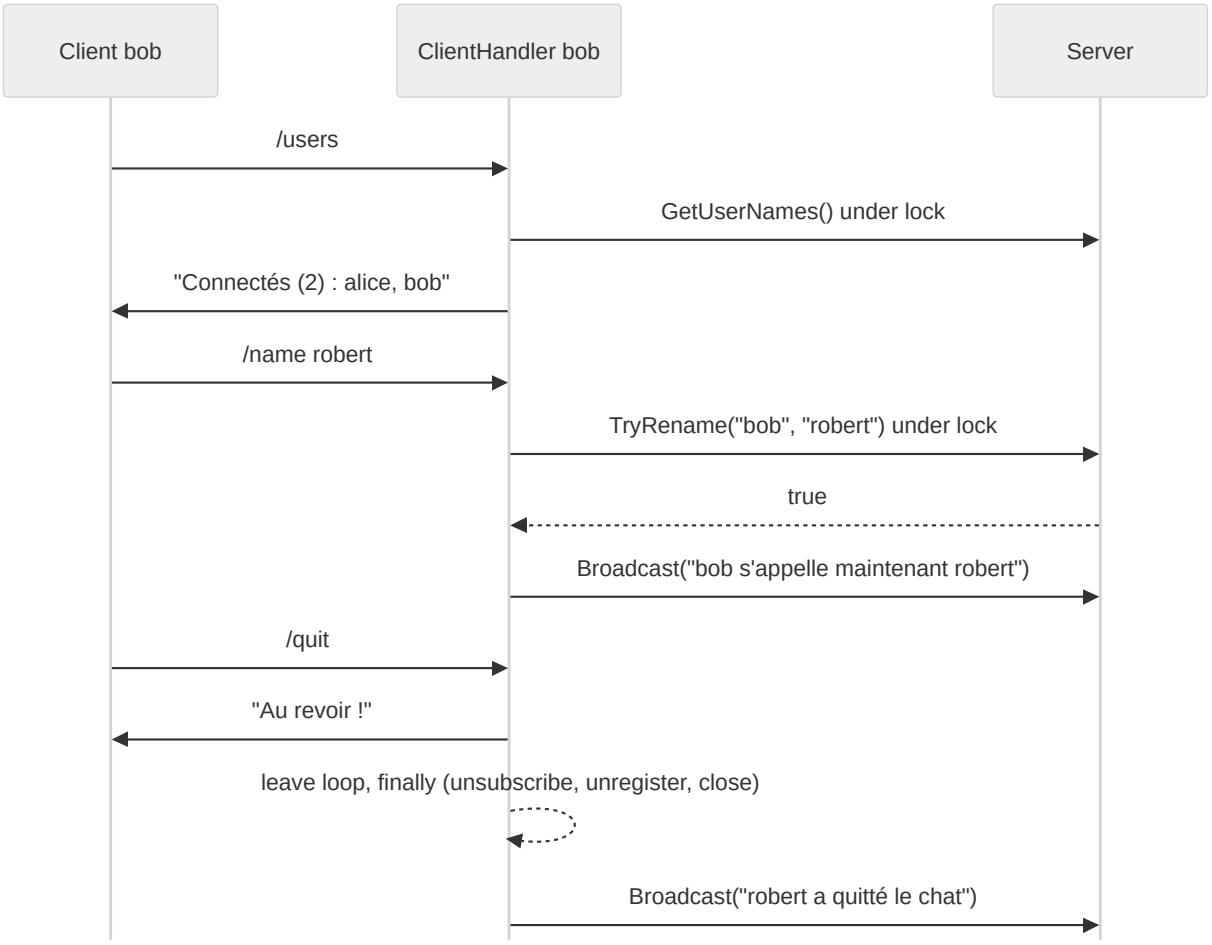




Private message, echo, and unknown-user error

b.6 Commands /users, /name, /help, /quit

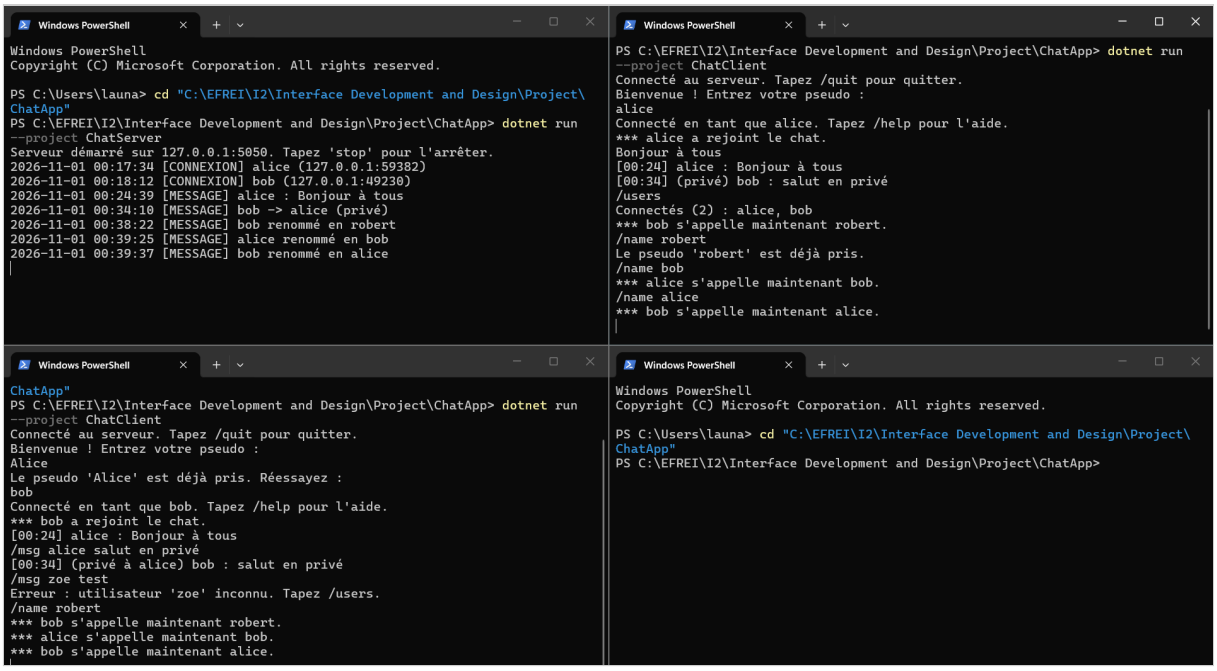
All commands are parsed on the server in `ClientHandler.HandleCommand`. `/users` returns a sorted copy of the dictionary keys taken under the lock. `/name` calls `Server.TryRename`, which checks uniqueness and replaces the key inside the same lock (no window where two users could take the same name), then announces the change to everyone. `/quit` answers “Au revoir”, leaves the read loop and lets the `finally` block do the cleanup. `/help` lists the commands.



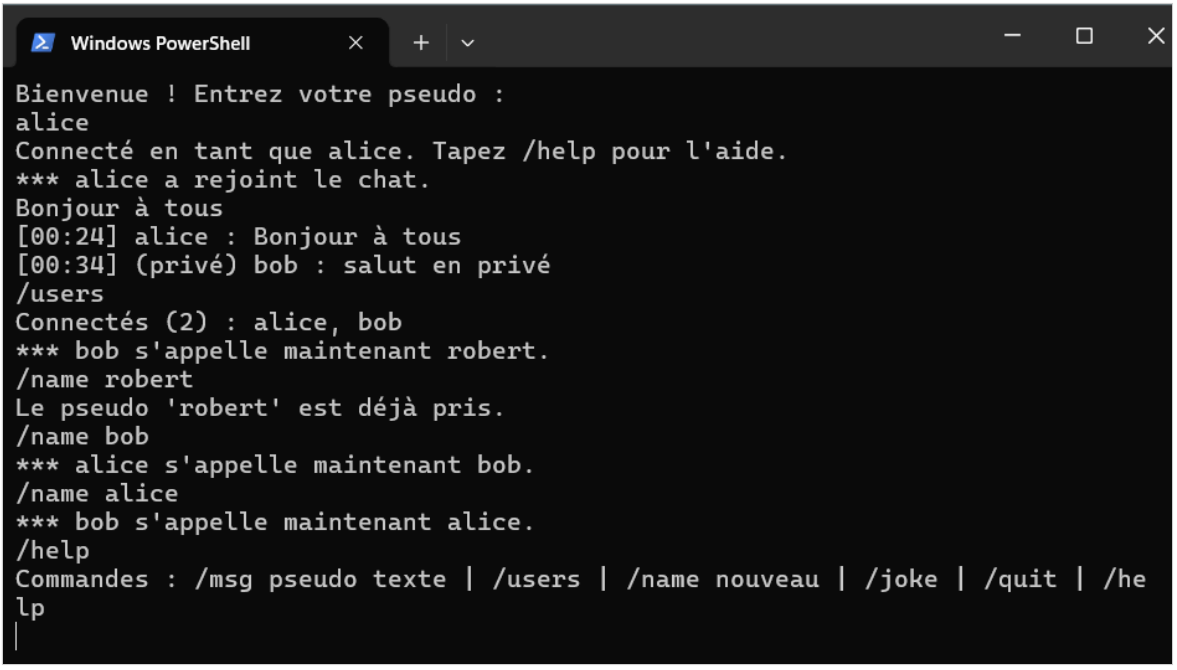

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\launa> cd "C:\EFREI\I2\Interface Development and Design\Project\ChatApp"
PS C:\EFREI\I2\Interface Development and Design\Project\ChatApp> dotnet run --project ChatClient
Connecté au serveur. Tapez /quit pour quitter.
Bienvenue ! Entrez votre pseudo :
alice
Connecté en tant que alice. Tapez /help pour l'aide.
*** alice a rejoint le chat.
Bonjour à tous
[00:24] alice : Bonjour à tous
[00:34] (privé) bob : salut en privé
/users
Connectés (2) : alice, bob
```

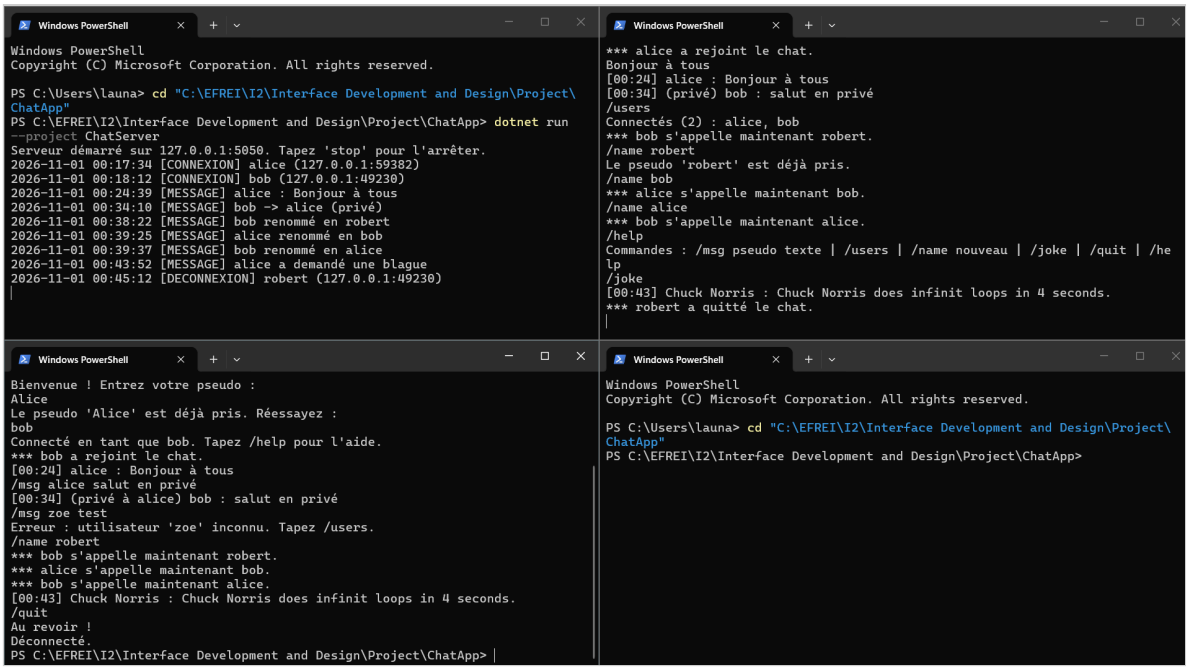
`/users`



`/name`, including a refused duplicate



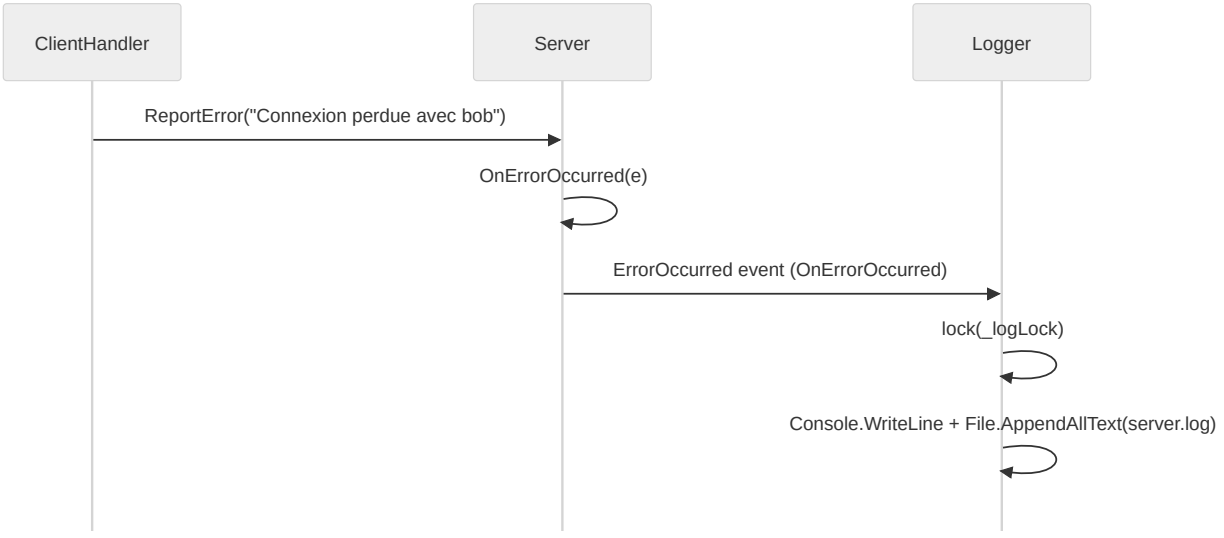
`/help`



`/quit` and departure announcement

b.7 Logging server activity (server 8)

The `Logger` uses exactly the same pattern as the broadcast: `Server` exposes four events of type `LogEventHandler` (`ClientConnected`, `ClientDisconnected`, `MessageReceived`, `ErrorOccurred`) raised through `protected virtual On…` methods. `Logger.Attach` subscribes to them. Every handler formats a time-stamped line with a category and writes it to the console and to `server.log` inside a lock, because events are raised concurrently by several client threads. The server code never writes errors with `Console.WriteLine` directly.



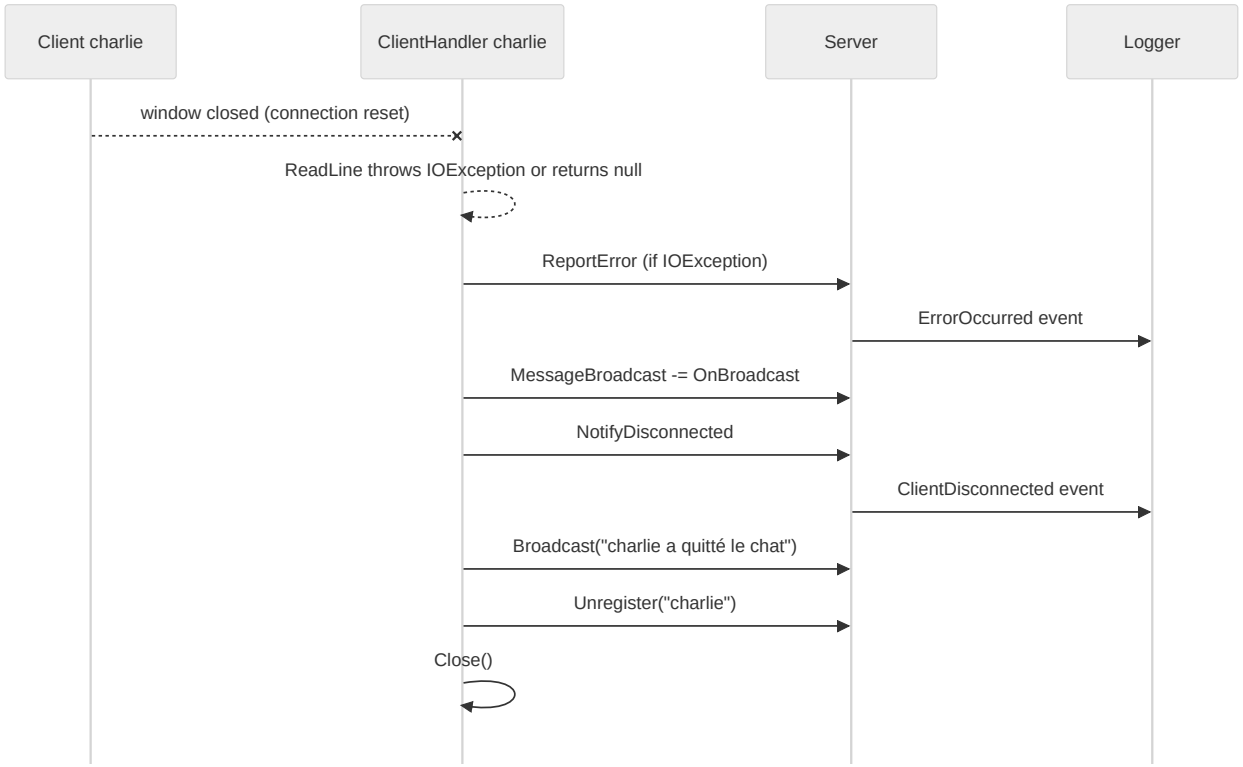
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

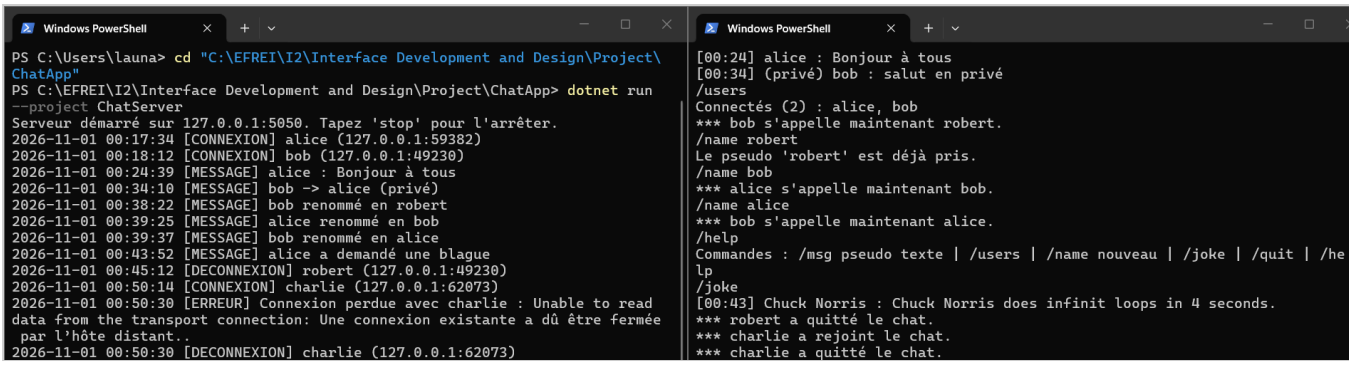
PS C:\Users\launa> cd "C:\EFREI\I2\Interface Development and Design\Project\ChatApp"
PS C:\EFREI\I2\Interface Development and Design\Project\ChatApp> Get-Content server.log -Encoding UTF8
2026-11-01 00:17:34 [CONNEXION] alice (127.0.0.1:59382)
2026-11-01 00:18:12 [CONNEXION] bob (127.0.0.1:49230)
2026-11-01 00:24:39 [MESSAGE] alice : Bonjour à tous
2026-11-01 00:34:10 [MESSAGE] bob -> alice (privé)
2026-11-01 00:38:22 [MESSAGE] bob renommé en robert
2026-11-01 00:39:25 [MESSAGE] alice renommé en bob
2026-11-01 00:39:37 [MESSAGE] bob renommé en alice
2026-11-01 00:43:52 [MESSAGE] alice a demandé une blague
2026-11-01 00:45:12 [DECONNEXION] robert (127.0.0.1:49230)
2026-11-01 00:50:14 [CONNEXION] charlie (127.0.0.1:62073)
2026-11-01 00:50:30 [ERREUR] Connexion perdue avec charlie : Unable to read data from the transport connection: Une conn
exion existante a dû être fermée par l'hôte distant..
2026-11-01 00:50:30 [DECONNEXION] charlie (127.0.0.1:62073)
PS C:\EFREI\I2\Interface Development and Design\Project\ChatApp>
```

Content of `server.log`

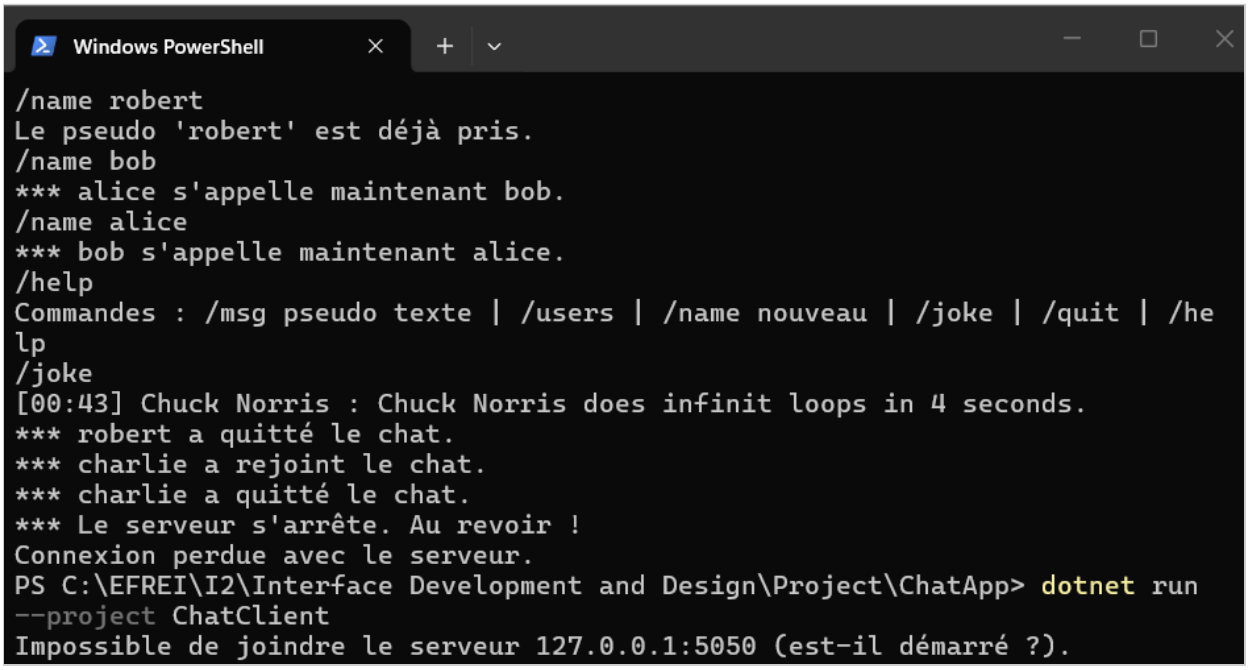
b.8 Disconnections and client robustness

On the server, `ClientHandler.Run` is wrapped in `try/catch/finally`: an abrupt disconnection raises an `IOException` (logged as an error), and the `finally` block always unsubscribes from the event, announces the departure, removes the user from the dictionary and closes the socket. A client that never finished the login is simply closed. On the client, a missing server is caught at connection time (`SocketException`). When the server closes the connection, the network thread detects the end of stream (or an `IOException`), prints “Connexion perdue avec le serveur.” and ends the process with `Environment.Exit`, because the main thread is blocked in `Console.ReadLine`.





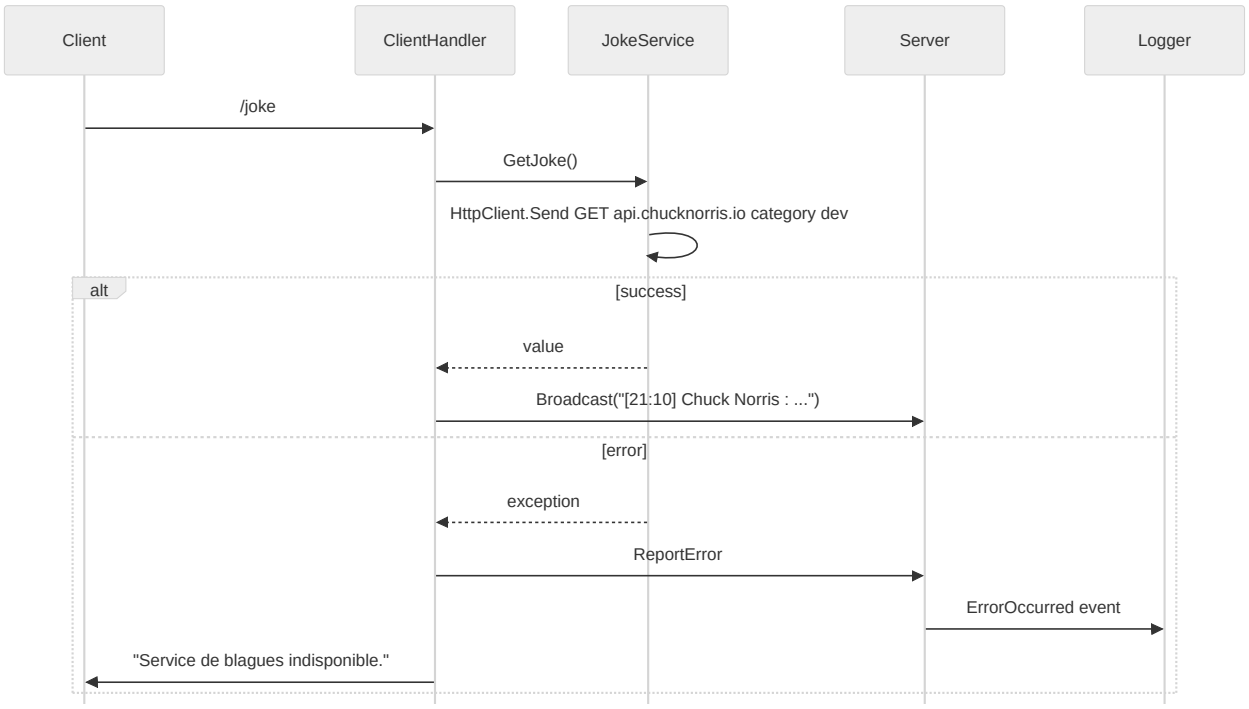
Client window closed abruptly: cleanup and announcement

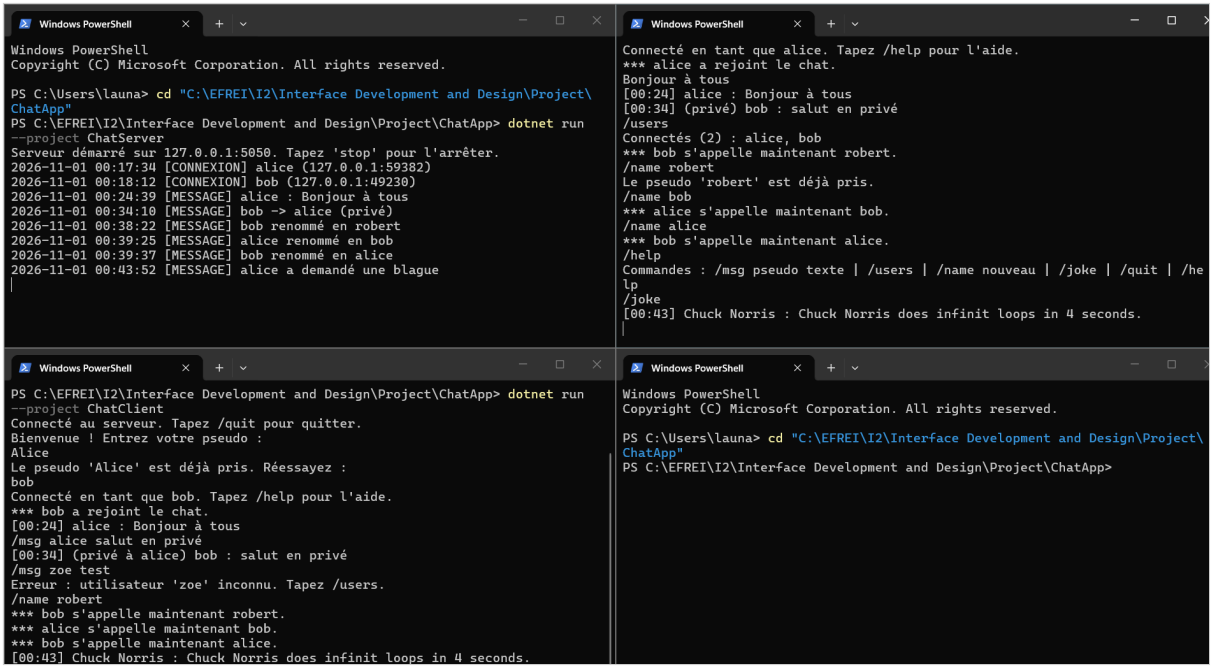


Client started while the server is down

b.9 Extension: /joke

`/joke` calls `JokeService.GetJoke`, which uses a single static `HttpClient` shared by the whole application and a synchronous `Send` (consistent with the thread-based design: the call blocks only the requesting client's thread). The request targets `/jokes/random?category=dev` to avoid explicit jokes. The JSON field `value` is read with `System.Text.Json` and the joke is broadcast to everyone. Any network or parsing error is logged and the requester receives “Service de blagues indisponible.”





`/joke` broadcast to every client

b.10 Known limitations

Limitation	Cause	Impact	Possible fix
A received line can be overwritten in the client window	On Windows, `Console.ReadLine` remembers where the input started and redraws the input from that position at the next keystroke. A line printed by the network thread	Display only: the message is correctly received (it is lost from the screen, not from the protocol).	Read the input key by key with `Console.ReadKey`, and share a lock with the network thread so it can clear the current input line, print the message, then redraw the input.

	while `ReadLine` is waiting can therefore be overwritten.		
Synchronous broadcast	The `MessageBroadcast` event invokes every subscriber on the sender's thread, and each `OnBroadcast` writes to a socket.	A slow client (full TCP buffer) slows down the sender of the message.	One outgoing queue and writer thread per client, or asynchronous writes.
One thread per client	Each connection holds a dedicated thread blocked on `ReadLine`.	Fine for a classroom chat, but does not scale to thousands of clients (memory and context switches).	`async` / `await` with asynchronous socket I/O.

c. Main parts of the C# implementation

c.1 Delegate and event for the broadcast (Server.cs)

The delegate defines the signature of the subscribers; the event can only be raised from inside `Server`. `?.Invoke` avoids a `NullReferenceException` when nobody is subscribed.

```
public delegate void MessageEventHandler(object sender, MessageEventArgs e);

public class MessageEventArgs : EventArgs
{
    public string Text { get; }
    public MessageEventArgs(string text) { Text = text; }
}

public class Server
{
    public event MessageEventHandler? MessageBroadcast;

    protected virtual void OnMessageBroadcast(MessageEventArgs e)
    {
        MessageBroadcast?.Invoke(this, e);
    }

    public void Broadcast(string text)
    {
        MessageEventArgs args = new MessageEventArgs(text);
        OnMessageBroadcast(args);
    }
}
```

Subscriber side (ClientHandler.cs): subscription after login, unsubscription in `finally`, and a writer lock because several threads may raise the event at the same time.

```
_server.MessageBroadcast += OnBroadcast;      // after a successful login
...
_server.MessageBroadcast -= OnBroadcast;      // in finally

private void OnBroadcast(object sender, MessageEventArgs e)
{
    Send(e.Text);
}

// Direct write to this client, protected by the writer's lock
public void Send(string text)
{
    lock (_writeLock)
    {
        try { _writer?.WriteLine(text); }
        catch (IOException ex)
        {
            if (!_server.IsStopping)
                _server.ReportError($"Envoi impossible à {Name ?? _endPoint} :
{ex.Message}");
        }
        catch (ObjectDisposedException) { }
    }
}
```

c.2 One thread per client (Program.cs, server)

```
// Accept loop in its own thread: the console stays free for 'stop'
Thread acceptThread = new Thread(() =>
{
    try
    {
        while (true)
        {
            TcpClient client = listener.AcceptTcpClient();
            ClientHandler handler = new ClientHandler(client, server);
            Thread thread = new Thread(handler.Run);
            thread.IsBackground = true;   // does not keep the process alive
            thread.Start();
        }
    }
    catch (SocketException ex)
    {
        // listener.Stop() unblocks AcceptTcpClient with an exception
        if (!server.IsStopping)
            server.ReportError($"Boucle Accept interrompue : {ex.Message}");
    }
});
```

c.3 Client list protected by a lock (Server.cs)

The dictionary is only accessed inside `lock (_clientsLock)`, a private object nobody else can lock. Check-and-insert and check-and-rename are atomic.

```
private readonly Dictionary<string, ClientHandler> _clients =
    new Dictionary<string, ClientHandler>(StringComparer.OrdinalIgnoreCase);
private readonly object _clientsLock = new object();

public bool TryRegister(string name, ClientHandler handler)
{
    lock (_clientsLock)
    {
        if (_clients.ContainsKey(name)) return false;
        _clients.Add(name, handler);
        return true;
    }
}

public bool TryRename(string oldName, string newName, ClientHandler handler)
{
    lock (_clientsLock)
    {
        bool sameUser = string.Equals(oldName, newName,
StringComparison.OrdinalIgnoreCase);
        if (!sameUser && _clients.ContainsKey(newName)) return false;
        _clients.Remove(oldName);
        _clients.Add(newName, handler);
        return true;
    }
}
```

c.4 Client life cycle and command handling (ClientHandler.cs)

```
public void Run()
{
    try
    {
        ...
        if (!Login(reader)) return;
        _server.MessageBroadcast += OnBroadcast;
        _server.NotifyConnected($"{Name} ({_endPoint})");
        _server.Broadcast($"*** {Name} a rejoint le chat.");

        string? line;
        while ((line = reader.ReadLine()) != null)
        {
            if (line.Trim().Length == 0) continue;
            if (line.StartsWith('/'))
            {
                if (!HandleCommand(line)) break; // /quit
            }
            else
            {
                _server.NotifyMessage($"{Name} : {line}");
                _server.Broadcast(Server.Format(Name!, line));   // [HH:mm] name : text
            }
        }
    }
    catch (IOException ex) { /* abrupt disconnection, logged */ }
    catch (ObjectDisposedException) { }
    finally
    {
        // Always executed, even after an abrupt disconnection
        _server.MessageBroadcast -= OnBroadcast;
        if (Name != null)          // only if the login succeeded
        {
            _server.NotifyDisconnected($"{Name} ({_endPoint})");
            _server.Broadcast($"*** {Name} a quitté le chat.");
            _server.Unregister(Name);
        }
        Close();
    }
}
```

```
private bool HandleCommand(string line)
{
    string[] parts = line.Split(' ', 3, StringSplitOptions.RemoveEmptyEntries);
    switch (parts[0].ToLowerInvariant())
    {
        case "/msg":
            ClientHandler? target = _server.Find(parts[1]);
            if (target == null || target.Name == null) { Send("Erreur : utilisateur ...
inconnu"); break; }
            target.Send($"[{DateTime.Now:HH:mm}] (privé) {Name} : {parts[2]}");   //
recipient's writer
            Send($"[{DateTime.Now:HH:mm}] (privé à {target.Name}) {Name} : {parts[2]}");
// echo
            break;
        case "/users": ...   // _server.GetUserNames()
        case "/name":  ...   // _server.TryRename(...) then Broadcast
        case "/joke":  ...   // JokeService.GetJoke() then Broadcast
        case "/quit":
            Send("Au revoir !");
            return false;   // leaves the loop, finally does the cleanup
        case "/help":  ...
    }
    return true;
}
```

c.5 Logger (Logger.cs and Server.cs)

```
public delegate void LogEventHandler(object sender, LogEventArgs e);

// In Server: same pattern as MessageBroadcast
public event LogEventHandler? ClientConnected;
public event LogEventHandler? ClientDisconnected;
public event LogEventHandler? MessageReceived;
public event LogEventHandler? ErrorOccurred;
protected virtual void OnErrorOccurred(LogEventArgs e) { ErrorOccurred?.Invoke(this, e); }
public void ReportError(string text) { OnErrorOccurred(new LogEventArgs(text)); }

// In Logger
public void Attach(Server server)
{
    server.ClientConnected += OnClientConnected;
    server.ClientDisconnected += OnClientDisconnected;
    server.MessageReceived += OnMessageReceived;
    server.ErrorOccurred += OnErrorOccurred;
}

private void Write(string category, string text)
{
    string line = $"{DateTime.Now:yyyy-MM-dd HH:mm:ss} [{category}] {text}";
    lock (_logLock)   // several client threads log at the same time
    {
        Console.WriteLine(line);
        File.AppendAllText(_path, line + Environment.NewLine);
    }
}
```

c.6 Stopping the server (Program.cs and Server.cs)

The handlers are copied under the lock, then warned and closed *outside* the lock: closing a socket makes the handler's `finally` call `Unregister`, which takes the same lock — doing it inside would modify the collection during enumeration.

```
// Program.cs
Console.WriteLine("Arrêt du serveur...");
server.IsStopping = true;
listener.Stop();   // no more new connections
server.StopAll("*** Le serveur s'arrête. Au revoir !");
acceptThread.Join(2000);
// wait (max 2 s) until every handler has finished its finally block
while (server.GetUserNames().Count > 0 && DateTime.Now < limit)
    Thread.Sleep(50);

// Server.cs
public void StopAll(string message)
{
    IsStopping = true;
    List<ClientHandler> handlers;
    lock (_clientsLock)
    {
        handlers = _clients.Values.ToList();
    }
    foreach (ClientHandler handler in handlers)
    {
        handler.Send(message);
        handler.Close();
    }
}
```

c.7 Client: two threads and robustness (ChatClient/Program.cs)

```
try { client = new TcpClient("127.0.0.1", 5050); }
catch (SocketException)
{
    Console.WriteLine("Impossible de joindre le serveur 127.0.0.1:5050 (est-il démarré
?).");
    return 1;
}

// Network thread: prints everything the server sends
Thread readThread = new Thread(() =>
{
    try
    {
        string? l;
        while ((l = reader.ReadLine()) != null)
            Console.WriteLine(l);
    }
    catch (IOException) { }
    catch (ObjectDisposedException) { }

    if (Volatile.Read(ref quitting)) return;
    // Server side cut: main thread is blocked on Console.ReadLine, so exit the process
    Console.WriteLine("Connexion perdue avec le serveur.");
    Environment.Exit(1);
});
readThread.IsBackground = true;
readThread.Start();
// Main thread: keyboard -> server (writer.WriteLine in try/catch IOException)
```